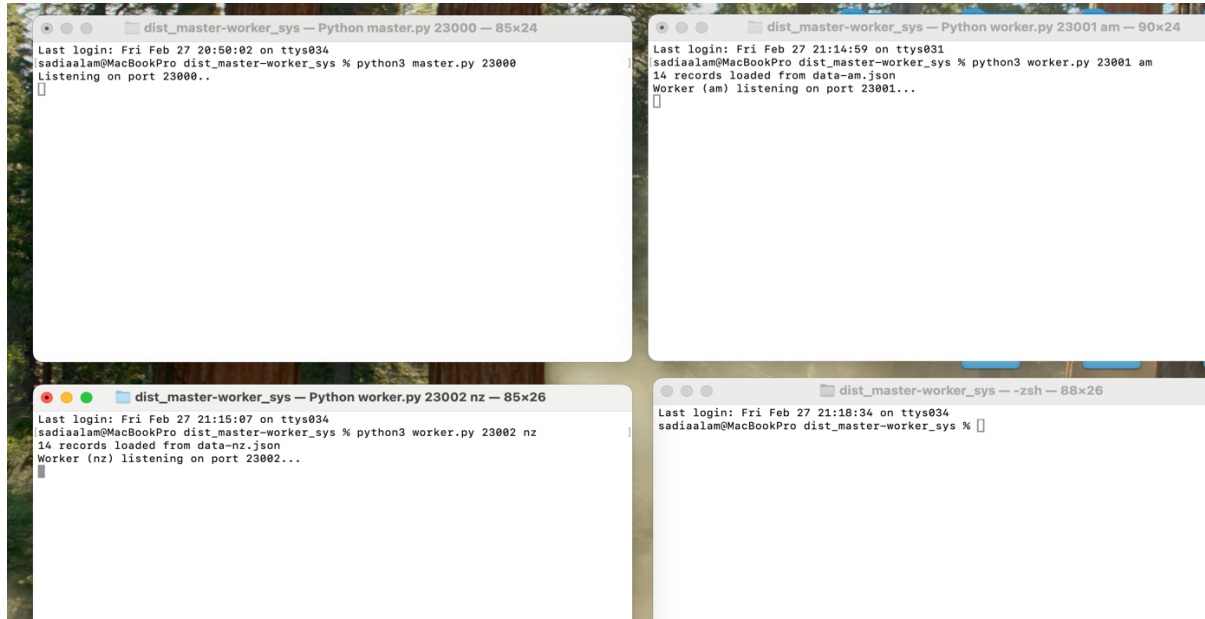


Sadia Alam

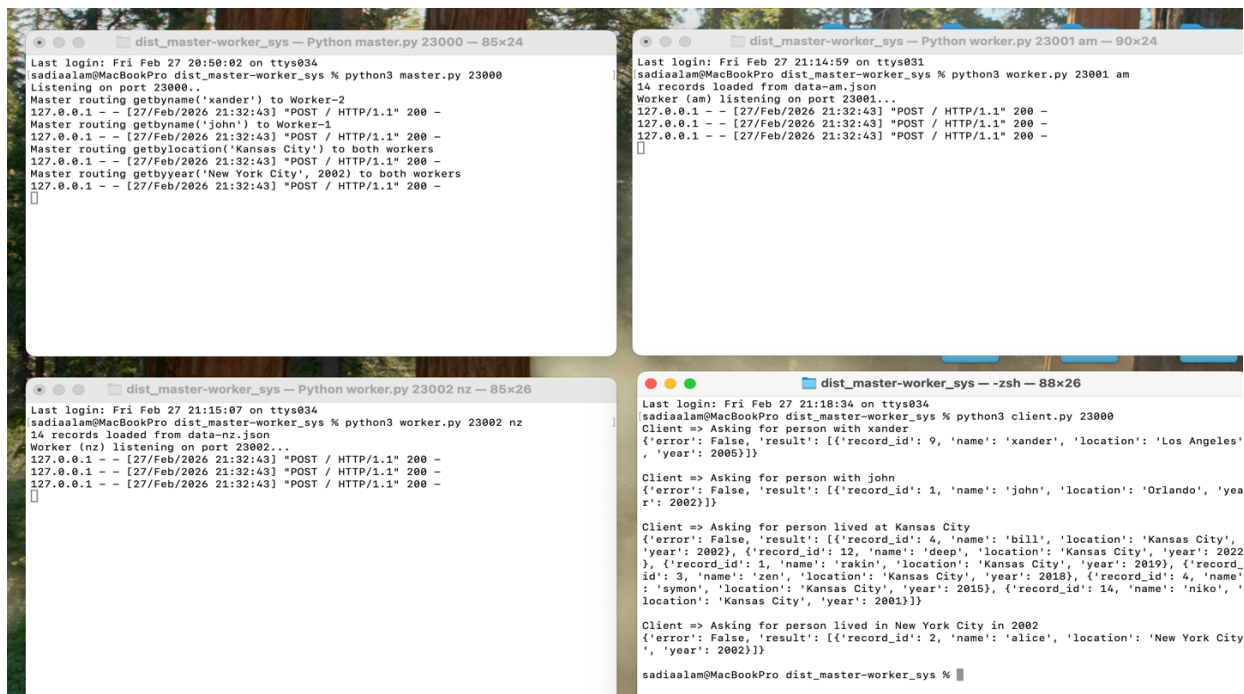
CS446: Distributed Computing System

02/27/2026

Distributed Master-Worker System



The master and worker servers are running successfully. The master and worker processes are listening to port 23000, 23001 and 23002 respectively. The worker processes load their relevant JSON file into python dictionary object.



When the client program is executed, master receives client's request and passes it to the correct worker and then forwards the matching records from worker process to the client. For example, in our program, the client calls the `getbylocation` function for a person's info who lives in Kansas City. The worker servers find and return the person's records that matches the location from their loaded JSON file. The master program processes the result received from the workers and sends it to the client. Thus, we see the records of the persons who are residents of Kansas City in the client terminal.

```

dist_master-worker_sys — Python master.py 23000 — 85x24
Last login: Fri Feb 27 20:50:02 on ttys034
sadiaalam@MacBookPro dist_master-worker_sys % python3 master.py 23000
Listening on port 23000..
Master routing getbyname('xander') to Worker-2
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
Master routing getbyname('john') to Worker-1
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
Master routing getbylocation('Kansas City') to both workers
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
Master routing getbyyear('New York City', 2002) to both workers
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
Master routing getbyname('xander') to Worker-2
127.0.0.1 - - [27/Feb/2026 22:10:11] "POST / HTTP/1.1" 200 -
Master routing getbyname('john') to Worker-1
Could not reach worker-1: [Errno 61] Connection refused
127.0.0.1 - - [27/Feb/2026 22:10:11] "POST / HTTP/1.1" 200 -
Master routing getbylocation('Kansas City') to both workers
Could not reach worker-1: [Errno 61] Connection refused
127.0.0.1 - - [27/Feb/2026 22:10:11] "POST / HTTP/1.1" 200 -
Master routing getbyyear('New York City', 2002) to both workers
Could not reach worker-1: [Errno 61] Connection refused
127.0.0.1 - - [27/Feb/2026 22:10:11] "POST / HTTP/1.1" 200 -

dist_master-worker_sys — zsh — 107x24
Last login: Fri Feb 27 21:14:59 on ttys031
sadiaalam@MacBookPro dist_master-worker_sys % python3 worker.py 23001 am
14 records loaded from data-am.json
Worker (am) listening on port 23001...
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
^CTraceback (most recent call last):
  File "/Users/sadiaalam/Desktop/dist_master-worker_sys/worker.py", line 141, in <module>
    main()
    ~~~~~^
  File "/Users/sadiaalam/Desktop/dist_master-worker_sys/worker.py", line 137, in main
    server.serve_forever()
    ~~~~~^
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/socketserver.py", line 235, in se
rve_forever
    ready = selector.select(poll_interval)
    ~~~~~^
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/selectors.py", line 398, in selec
t
    fd_event_list = self._selector.poll(timeout)
KeyboardInterrupt
sadiaalam@MacBookPro dist_master-worker_sys %

dist_master-worker_sys — Python worker.py 23002 nz — 85x26
Last login: Fri Feb 27 21:15:07 on ttys034
sadiaalam@MacBookPro dist_master-worker_sys % python3 worker.py 23002 nz
14 records loaded from data-nz.json
Worker (nz) listening on port 23002...
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Feb/2026 21:32:43] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Feb/2026 22:10:11] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Feb/2026 22:10:11] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Feb/2026 22:10:11] "POST / HTTP/1.1" 200 -

dist_master-worker_sys — zsh — 104x26
in', 'location': 'Kansas City', 'year': 2019}, {'record_id': 3, 'name': 'zen', 'location': 'Kansas City',
'year': 2018}, {'record_id': 4, 'name': 'symon', 'location': 'Kansas City', 'year': 2015}, {'record_id
': 14, 'name': 'niko', 'location': 'Kansas City', 'year': 2001}}]

Client => Asking for person lived in New York City in 2002
{'error': False, 'result': [{'record_id': 2, 'name': 'alice', 'location': 'New York City', 'year': 2002}
]}

sadiaalam@MacBookPro dist_master-worker_sys % python3 client.py 23000
Client => Asking for person with xander
{'error': False, 'result': [{'record_id': 9, 'name': 'xander', 'location': 'Los Angeles', 'year': 2005}
]}

Client => Asking for person with john
{'error': False, 'result': []}]

Client => Asking for person lived at Kansas City
{'error': False, 'result': [{'record_id': 1, 'name': 'rakin', 'location': 'Kansas City', 'year': 2019},
{'record_id': 3, 'name': 'zen', 'location': 'Kansas City', 'year': 2018}, {'record_id': 4, 'name': 'symo
n', 'location': 'Kansas City', 'year': 2015}, {'record_id': 14, 'name': 'niko', 'location': 'Kansas City
', 'year': 2001}}]

Client => Asking for person lived in New York City in 2002
{'error': False, 'result': []}]
sadiaalam@MacBookPro dist_master-worker_sys %

```

From the screenshot, we can see that I killed the worker 1 server. When the client program is executed, the master can't send request to worker 1 but sends the request to worker 2. Therefore, in the client terminal, we see the matching records the master routed from worker 2.

It's quite interesting that even if one of the servers are killed, a failed connection doesn't crash our whole program. When one server is killed the master server stays alive. Master server works as a middleman here. When one worker server is down, it sends the request to the other active server and returns whatever results it could get from the running worker. For example, in our program, the client requests for person's records with the name 'John'. But the worker 1 server handles the info for this request. Instead of crashing our program it returned an empty list. The master process doesn't store any data. It connects between the client and worker processes. If the master is down, then no requests can be processed even if the worker servers are active. Also, I noticed how the work is evenly balanced between the worker processes. If worker 1 is killed, the queries with the names that start with letter n-z don't have to suffer. For example, in our program, when the client requests for the records of the residence of Kansas

City the query is sent to both workers. Since worker 1 is down, the matching records from worker2 gets displayed.